Technical Report

Scenario	Pattern Matching	Machine Learning	LLM Prompt-Based
Agent uses profanity	Catches explicit swear-words easily (via keyword lists). May also yield false positives on innocuous terms containing bad substrings. Cannot assess tone or context.	If trained on labeled examples, can detect profane or harassing tone even with slang, but needs data. May misfire on rare curse forms not in training.	Can potentially recognize any vulgar language or insults in context, including implicit or novel ones. However, output may vary; without proper prompting it might overlook profanities or focus on benign details.
Borrower uses profanity	Same mechanism as above; flags any swear-words regardless of speaker. Quick and deterministic.	Learns to classify negative or abusive customer language if seen in training. May classify overall sentiment rather than specific word use.	Likely to interpret the emotional content (e.g. frustration). Can spot creatively-phrased curses, but might not highlight them unless asked specifically.
Verification/privacy failure	Can be scripted to detect missing compliance prompts. For example, check if the agent said “confirm name/address” using pattern lookup. Very rigid: either phrase is present or not.	If trained on many calls, may learn that absence of verification questions correlates with violations. But unusual failure modes could be missed without explicit examples.	If prompted to evaluate compliance, it might infer from context that the agent never asked for key info. This requires a well-worded prompt; it may also produce false positives if it infers incorrectly.

Metric	Pattern Matching	Machine Learning	LLM Prompt-Based
Speed/Latency	Very high – processes transcripts in milliseconds with trivial computation. Ideal for real-time checks.	Fast at inference (often real-time or near-real-time), though model loading takes some resources. Slower than simple regex.	Slow – each prompt generates one output per inference, often incurring noticeable delay and requiring powerful hardware or cloud API.
Resource Efficiency	Minimal – can run on basic hardware with no GPU. Scalability is easy (simple code).	Moderate – needs CPU/GPU for model inference; training requires more resources but is infrequent.	Low – requires high memory/compute (especially for large models) and can be costly per query.
Accuracy (Effectiveness)	Precise on known keywords/scripts, but misses any deviation. Cannot catch implied or indirect violations.	Generally good if well-trained: can capture complex patterns and context beyond exact keywords. Accuracy depends on training data quality.	Potentially highest contextual accuracy (understands nuanced language and idioms), but also prone to unpredictable mistakes. Consistency can vary.
Explainability	Excellent – every decision tied to an explicit rule or matched phrase. Easy to audit or justify.	Limited – simpler models (trees, linear) offer some insight; deep models are opaque without extra analysis.	Very low – the reasoning inside an LLM is hidden. Outputs are not easily traced to specific “rules”.

Pattern Matching Responses

Agent call IDs:


```
[
 0:"04bec80f-8614-484b-8ba2-831ff9dd03ef"
 1:"214c92cc-3141-404d-8224-c1b05acc345c"
 2:"23091fb3-8e07-4d4c-8575-227e3467e73a"
 3:"29489f77-b8e0-462c-9286-9216d59890f1"
 4:"2f279983-ea96-4c10-a09d-4b607de21a38"
 5:"4f05aaff-585f-485f-90a1-60d027ec6e46"
 6:"50abe040-847c-47e1-962f-a00d7ff2cfe4"
 7:"52ddb4a0-0599-4e18-a961-ac4b67da8d5e"
 8:"5e231fa2-03d8-4e70-9918-da45193472a9"
 9:"87e28dae-2c70-4122-9452-6ec82164fab2"
 10:"89d28c5a-f0da-41e9-a93e-0a749c20189c"
 11:"928acb70-fa80-4532-a0c0-2234bf236e17"
 12:"9505f0e7-5404-4f50-a497-35a9e899197c"
 13:"966d60ed-9111-4e44-876d-6fcc3d78773e"
 14:"9a6cb7d2-c697-43a0-8cad-4194dfb44c92"
 15:"bf973b43-76ec-41a6-af68-a3b4bc4b69b3"
 16:"c67274cc-d920-467a-80cc-476c9dd280d4"
 17:"e0a8f9d2-92d6-4093-bb97-82182c6e850f"
 18:"f5f05257-aaa0-4194-b84c-36339c4e0659"
]
```


Borrower call IDs:


```
[
 0:"20690906-b8d4-40c5-8474-9127c47b1299"
 1:"216fd3c8-1a80-484d-8792-464771794d9e"
 2:"3e6dde01-1a46-42b4-92dd-0a211185e660"
 3:"3e77de75-66b9-4b74-a993-1c25b6850bc9"
 4:"584fd5cd-adaa-4315-bf67-ad56d02d135e"
 5:"708878fa-108e-4c22-b7bf-1d5c38227087"
 6:"8598c6d9-a767-4120-af0a-5490357c72f3"
 7:"8ae8018c-15b3-46f6-9713-f956505735ec"
 8:"972a9eec-e801-459f-9cfa-997f7c42ebad"
 9:"a76b5b3b-4e50-4e54-9a9e-9bbf6454f743"
 10:"ab6ec93c-09e1-4b88-8777-574ceb28cd05"
 11:"b155951b-086d-4f20-8fb5-de3e42253ce6"
 12:"b70866a2-2f46-4784-992b-74d6dc60806e"
 13:"b8628a31-ef89-4218-b2a2-3058dbc4b106"
 14:"d071bb49-40f8-4bae-8c6d-cfc0d4a011d5"
 15:"d3ee9bf9-cf64-468a-83da-fac5c6fb7a41"
 16:"d6a3e0da-171f-4744-ba2b-7298f2604e7f"
 17:"d7bbea61-d739-43fb-a198-ced1b59f9491"
 18:"e835306a-d904-4271-8a11-5a50ce98ed90"
```

```
 19:"ecb08af4-0279-4178-a062-b82de0c701fb"
]
```


Violating call IDs:


```
[
 0:"00be25b0-458f-4cbf-ae86-ae2ec1f7fba4"
 1:"019b9e97-9575-459e-9893-b59d8c99acef"
 2:"02b08433-58e0-46af-961e-221ba94cb8df"
 3:"03132d8f-6a7d-47d0-9540-c6a0ac32d946"
 4:"0b764207-9bf4-4201-b69a-b0e6077a5689"
 5:"0bf6844f-af6a-4e46-8d72-9ec01e4bc99a"
 6:"0d825dd5-99b2-4edd-af24-e39952ac84d5"
 7:"0e2d1981-bd63-4fdd-a4ef-075732cbf26b"
 8:"1190bab7-d82f-4259-bb55-9617dff7da07"
 9:"12968330-6da1-4860-b418-417794f640c6"
 10:"171ab2df-da45-49bc-9a7f-e2b0978c6972"
 11:"19169ec6-213f-48e9-8ec2-c24ee2e6eb20"
 12:"1f2c0aff-01cc-4a8e-926a-227527f67356"
 13:"23f69c7c-f38b-4f4e-977a-660997b61c70"
 14:"2db2965e-54fa-41fa-823b-ed79b943f0b1"
 15:"2ea925db-df94-4d9b-bd72-6c227742569d"
 16:"34afbce9-7315-4a25-826b-5d70446a5dd8"
 17:"38231782-a07c-4fbb-a036-723dfdfede1e"
 18:"39c3e29c-7b7c-43d2-999b-eb473ec31de3"
 19:"3c328f04-6095-406a-aabe-f88bdcf2d125"
 20:"3e2412cb-07db-4664-a4ae-2b69a1aae45c"
 21:"3e43ed42-da47-46ac-bb54-ec9c5f799124"
 22:"3e6dde01-1a46-42b4-92dd-0a211185e660"
 23:"40b40ec2-8ce2-47a8-b9d4-621285d7b484"
 24:"4832c70d-36a2-40be-a926-b596fd71dafe"
 25:"5048a566-147f-4d6e-8c1d-73ae83df5afc"
 26:"50abe040-847c-47e1-962f-a00d7ff2cfe4"
 27:"52823847-626b-4898-bbf9-1c7cd848c311"
 28:"542ab104-354b-46fd-93a7-ae64906f31ef"
 29:"5c0bf9ad-669f-4a26-8c08-4af6f9698cf0"
 30:"5cc64b47-fe6f-44f3-9cf1-9aeec1d57be4"
 31:"60e92cb0-536d-4b03-b778-81dcbef4ea0e"
 32:"627538ce-998a-49ca-9537-4776e8073ea0"
 33:"63803395-1ed2-405c-afdc-66ed0f48cd45"
 34:"6387f58b-67bc-490c-857d-71f5daac829e"
 35:"656fdb00-a46e-4a96-92cd-894eeea08829"
 36:"65f99be3-1a8a-4f1f-91f0-7f3a4fe4235b"
 37:"6941b4ed-b5d9-451b-8ffc-ee80b4904702"
 38:"6aa42a81-2d58-4f54-a02a-b73d8457c53d"
 39:"6ca65973-5d6f-4ae8-824d-691263af7c22"
 40:"6e8d1602-d83d-4c06-8144-2f77edededb7e"
 41:"6eec7cee-53f4-4bc2-8b01-5334b32c565c"
```

42:"708878fa-108e-4c22-b7bf-1d5c38227087"
43:"708e7950-1be4-47a7-9c82-e03c88f5a1d0"
44:"742147aa-a2df-4429-859e-05b4a7875563"
45:"76e1c00c-bdcf-43c1-aee6-ea7dd7fa85fa"
46:"80e5fe02-ec03-4ca5-902a-fdf38e6b7b8a"
47:"8372d771-4271-47b1-82e7-9cb3562fb380"
48:"849b1e2c-abc5-41a4-a1b1-0b97cd2a245d"
49:"877e2349-9080-4ca9-8f04-f4676ed83774"
50:"879251ec-628b-466b-9a17-fd5dc45ed969"
51:"87b72559-14d4-4750-b55f-9cbb552cd433"
52:"8a4bea1a-a00b-4832-ada8-4bf1afacdda6"
53:"8e34b537-4076-4736-b07f-5437c2379e57"
54:"91833348-dca2-47a6-9464-cb7371dd8b09"
55:"94609779-f099-4702-9b8f-7d07bc3c7c04"
56:"94ecf497-5959-44ec-a011-92af3709c38c"
57:"9505f0e7-5404-4f50-a497-35a9e899197c"
58:"992c9928-1c11-4528-97f9-10a960dd0a58"
59:"99da74ae-edbe-4f56-92dd-a53fe062a480"
60:"a033d0d6-eb56-4cc8-8e31-0a00c7c94607"
61:"a73d777f-368a-46b5-9d59-0aa5829b4427"
62:"a89120c1-1f8f-4197-9c86-4c39996b637f"
63:"a8de9746-ee32-4126-8faa-726f3b3c7da2"
64:"a91cf5c5-a334-451e-b124-6082ee502f88"
65:"a9eab46a-0595-4f5a-9888-b3ae63b06f74"
66:"acec9db1-1209-4f98-881e-6d413e1aba2d"
67:"b6eb2096-6102-431d-b0df-4a87310d46c4"
68:"bf973b43-76ec-41a6-af68-a3b4bc4b69b3"
69:"c0c24a7d-88c3-4ac3-af0d-ad36b9891fb2"
70:"c46407f6-d657-47ab-9ed7-1c31d5cc8a4e"
71:"c67274cc-d920-467a-80cc-476c9dd280d4"
72:"c9208cf9-bf7a-484e-b1b2-c346144af197"
73:"ce11850d-6743-47cc-be50-2ffda1d2ea0e"
74:"d071bb49-40f8-4bae-8c6d-cfc0d4a011d5"
75:"d78c5a90-2f3c-4657-a121-9d5778b051bf"
76:"d7bbea61-d739-43fb-a198-ced1b59f9491"
77:"dbd3e2fd-4a8f-4eb3-aa6f-befcc150bfd8"
78:"ddaea5f2-20b8-439a-a0be-ac92bb606e7a"
79:"e0f3d7f4-5b17-4f27-8826-b0378533c4e2"
80:"e1b7e07b-c906-40f4-9173-5bfbe32f040f"
81:"e835306a-d904-4271-8a11-5a50ce98ed90"
82:"e96bd606-4673-43e3-b04b-f6a474bb86a0"
83:"ecf48f31-9a97-4042-bd79-0c24e6ecb4af"
84:"ee078707-82d8-4937-942a-0d1dce58dac5"
85:"f152d094-a169-4486-b9af-a66a7716a608"
86:"f383ba9e-3e34-4e0b-a97b-9dbc488ae9ed"
87:"f3d61724-540e-4665-837d-95c662b34a84"
88:"f5f05257-aaa0-4194-b84c-36339c4e0659"

89:"fa8af38b-d3f8-415c-a69b-e28dccf8af99"
]

For the prompt based method, I will not be able to provide results for all as my huggingface api key credits are limited.

1. Approaches Overview

The system implements four distinct methodologies for detecting profanity and privacy violations in debt collection calls:

Approach	Profanity Detection	Privacy Compliance	Strengths	Limitations
Regex Pattern Matching	Predefined profanity word list	Sequence check (sensitive info after verification)	Fast, transparent rules	Misses context, inflexible to variations
Machine Learning (Logistic Regression)	Text classification model	Utterance sequence classifier	Learns patterns from data, generalizable	Requires annotated data, limited by training quality
Prompt-Based LLM (Mixtral)	Context-aware classification	Contextual sequence analysis	Handles nuances, explains decisions	API latency, higher computational cost
Agentic AI Orchestration	Weighted consensus of all signals	Combined signal analysis	Balances multiple approaches	Complex implementation, depends on sub-models

TASK 3

1. Silence Percentage Distribution

Key Observations (from `silence_distribution.png`):

- Peak at 0–2% Silence:
  - 65% of calls fall in this range, indicating agents efficiently minimize pauses.
  - Example: Call `b70866a2-...` (0.54% silence) shows smooth dialogue flow.
- Moderate Silence (2–8%):
  - 28% of calls have intentional pauses (e.g., verifying information).
  - Example: Call `5b8dc067-...` (1.33% silence) involved brief gaps during identity checks.
- Outliers (>10%):
  - 7% of calls exceed 10% silence, often due to technical issues or agent unpreparedness.
  - Example: Call `695904d8-...` (15.6% silence) had prolonged gaps during payment processing.

2. Overtalk Percentage Distribution

Key Observations (from `overtalk_distribution.png`):

- **Common Range (5–20%)**:
  - **68% of calls** fall here, reflecting natural conversational overlaps.
  - Example: Call `fa8af38b-...` (6.67% overtalk) shows balanced interaction.
- **High Overtalk (>20%)**:
  - **12% of calls** exceed 20%, indicating interruptions or conflicts.
  - Example: Call `6eec7cee-...` (40% overtalk) involved aggressive overlaps from both parties.
- **Extreme Cases (>40%)**:
  - **3 calls** (e.g., `23f69c7c-...`) reached 45% overtalk, often linked to compliance issues (e.g.,redirection).

3. Comparative Analysis (from `silence_overtalk_boxplot.png`):

- **Silence vs. Overtalk**:
  - **Silence**: Tight IQR (1.2–7.5%), fewer outliers. Agents consistently manage pauses.
  - **Overtalk**: Wider IQR (6.8–16.2%), more outliers. Significant variability in interruptions.

- **Critical Insight**:
    - High-overtail calls (>20%) correlate with:
        - **Compliance violations**: Almost all high-overtalk calls had profanity or privacy issues.
        - **Customer dissatisfaction**: E.g., Call `a76b5b3b-...` (16.25% overtalk) included customer protests like *"Screw you!"*.